Score Following on Sheet-Music Images

Findings, corrections, and current state

14 August 2026

Contents

How to read this

Every factual claim in these documents is tagged:

- **[V]** verified directly by me in code, data, or a job log
- **[A]** reported by a research agent and **not** independently re-checked
- **[E]** an estimate or inference, explicitly not a measurement

That separation exists because several claims in this project turned out to be wrong — including two I relayed before checking. The tags let you weight each statement appropriately.

Current standing, at a glance

model	room (pct@0.5s)	whose
published CUNet — our own architecture family	22.4	literature
MM-Loc — best non-detection model published	58.5	literature
our best (R2r_realir)	**56.6**	ours
cyolo_sb — the bar, reproduced by us	**79.9**	CPJKU's
CODA — claimed SOTA	88.3	not comparable (see audit)

The headline caveat, which applies to every number above: a piece-level cluster bootstrap gives this benchmark an effective sample size of **25 pieces, not 4415 onsets**, so its minimum detectable difference is roughly **10 points**.

Master plan — score following on sheet-music images

Written 2026-08-08. Supersedes `PLAN_2026-08-06.md` and the track docs (`REAL_AUDIO_TRACKS.md`, `HYBRID_MODELS.md`), which remain valid as detail but were written against numbers since corrected.

Every claim below is tagged: **[V]** verified by me directly in code, data, or a job log · **[A]** reported by a research agent, not independently re-checked · **[E]** estimate or inference, explicitly not a measurement.

---

0. One-paragraph summary

We build image-conditioned score followers: given live audio and a sheet-music image, predict the pixel position frame by frame. Our real-audio number is **45.6** pct@0.5s on MSMD-Rec `room`; the reproducible bar is **cyolo_sb = 79.9**. Roughly two months of work assumed that gap was *acoustic* — that room reverberation destroys the audio signal. **That assumption is now falsified.** A stock automatic music transcriber recovers 91–95% of onsets from the identical recordings, and the room costs it essentially nothing. The information is present; our architecture cannot hold onto it. Separately, an audit of the current SOTA claim (CODA, ISMIR 2026) found it is measured under an undisclosed oracle and a different aggregator. So the work now has **two deliverables**: an evaluation/measurement paper that is essentially finished, and a model track with a newly-localised target.

---

1. Where we actually stand

1.1 The scoreboard (pct@0.5s, MSMD-Rec `room`, micro-aggregated)

system	room	note
CODA (ISMIR 2026)	88.3 **claimed**	macro + oracle layout boxes — see §3.1
cyolo_sb_a	86.5	needs “+A” data we do not have; not reproducible
cyolo_sb — the real bar	**79.9**	**[V]** we reproduced it exactly
CYOLO	71.2	
MM-Loc (Dorfer-style bucketed softmax)	58.5	same lab, same data, same audio tower
ours (R3: MERT + pitch-aux + belief filter)	**45.6**	
CUNet — published, our own architecture family	**22.4**	

[V] Our reproduction passes `--only_onsets` (`eval_cyolo_sota_cpu.sh:43,49`) and lands on `63.0 @0.1s / 79.9 @0.5s`, matching CODA’s copied baseline row `.630 / .799` exactly. So the bar is self-reproduced, not trusted — and the onset-only denominator is confirmed identical on both sides.

Two things this table says that were invisible for months:

- **We are extending a weak baseline, not failing at a strong one.** Published CUNet — the exact dense-heatmap + soft-Dice family we build on — scores **22.4**. R3 is 45.6. We have already roughly *doubled* the published version of our own architecture.

- **The target was misread by 17 points.** 63.0/70.6 are the **≤0.1 s** column of Frontiers Table 4; every number we report is ≤0.5 s. See `BASELINE_CORRECTION.md`.

1.2 The strongest architectural signal we have

[V] Within one paper, one lab, one dataset, one audio tower: **MM-Loc 58.5 vs CUNet 22.4** on room. A **36-point swing attributable to the output parameterisation alone** — bucketed softmax classification over position versus dense soft-Dice heatmap. Corroborated externally by the OOD-segmentation literature (Galdran et al.): soft-Dice wins in-distribution and loses out-of-distribution, which is exactly the synth→real transition.

This is stronger evidence than anything in `HYBRID_MODELS.md`, and it points at the output layer, not the audio front end.

---

2. The decisive new result: the room is not the problem

[V] `results/amt_bridge_eval-437566.log`, 25 real recordings, two AMT models, two microphones on the **same take** — so the room is isolated with the performance held exactly fixed.

	room	di-left	Δ (room − di-left)
kong_stock onset-F1 @50 ms	0.9116	0.9124	**−0.001**
kong_stock onset-F1 @100 ms	0.9546	0.9903	−0.036
edwards_robust @50 ms	0.9441	0.9259	**+0.018**
edwards_robust @100 ms	0.9864	0.9909	−0.005

The room costs a transcriber ~0 points at 50 ms, and the reverb-augmented model is *better* on room than on the direct pickup. Our image models lose ~35 points on those same files.

Consequence. The prior story — “82% of the 34-point gap is acoustic” — is **wrong**. Note-level information survives the room nearly intact. The failure is representational: our front end plus dense-heatmap output is brittle to the domain shift, not starved of signal. This is a far more actionable diagnosis, and it aligns with §1.2 pointing at the output parameterisation.

2.1 Guards that make these numbers trustworthy

- **[V]** Oracle ceiling recomputed in-harness: **100.00** pct@0.5s at latency 0 — the score-note → pixel → onset-frame round-trip is lossless, so no AMT number is confounded with a coordinate bug. (94.79 at latency −0.050, independently matching an earlier pass’s 93.4.)
- **[V]** Constant wav-vs-MIDI offset measured at −0.03 to −0.04 s across all four conditions — under one frame at 20 fps. Systematic but harmless at 0.5 s.

2.2 What these numbers are NOT

- **[V]** Offline DTW over the transcriptions scores **98.06** on room. **This does not beat cyolo_sb.** It is non-causal (score following must be online), and near-tautological here: score MIDI is byte-identical to performance MIDI (Disklavier playback), so there is no tempo deviation for DTW to fight.
- **[V]** The **online** matcher is where it collapses: greedy scores **10.7** (room) / 26.7 (di-left) with median error of **8–13 seconds**. It loses the pointer and never recovers. *That*, not transcription, is the hard part of a causal decomposed tracker — and it is now precisely located.

---

3. Findings register — what ~10 agent investigations established

3.1 CODA's SOTA claim is not like-for-like [V, in full]

`CODA_COMPARABILITY.md`. Audited at commit `dba9829`.

- **CODA is handed the page layout — at training time too.** `system_boxes` and `bar_boxes` are **required positional arguments** of `forward()` (`coda_model.py:314-317`), passed straight from the MSMD npz to the training call site (`train.py:173-177`). The loss has exactly three terms — system CE over given candidates, bar CE over given candidates, note MSE — with **no box regression, no objectness, no IoU**. No detection path exists in the release at all; the inherited YOLO builder is dead code. Its own docstring: *"No anchors, no NMS, no objectness — pure classification over known candidates."* **CODA picks 1 of N given regions; CYOLO must produce the region.** The paper does not disclose this.
- **Aggregation swap.** CODA reports **macro** (`evaluate_batch.py:179-183`); CYOLO reports **micro** (`eval.py:76-83`). Its own Table 1 (macro) and Table 2 (micro) disagree with each other.
- **Not reproducible.** No released weights; the real-audio Setting II ships no data, no split file, and no code path (`finalize_run.py` hard-codes the synthetic 94/66/28 counts). Baselines are copied from Henkel & Widmer, not re-run.
- **Not a defect:** thresholds, units, and the onset-only denominator are identical on both sides.

3.2 Aggregation alone is worth +7.3 points [V, on our own data]

`AGGREGATION_FINDING.md`. Same R3 checkpoint, three aggregators:

aggregation	R3 room
MICRO (what we and every baseline report)	**45.4**
MACRO over 25 pages	46.8
MACRO over 16 pieces — CODA's aggregator	**52.7**

Cause: Chopin Op.9 is **29.8% of the micro metric** but **6.2% macro-by-piece**, and scores 14.6 where the rest scores 57.0. Macro quietly down-weights the hardest 30% of the benchmark by ~5×. **[E]** If CODA's 88.3 is macro and it degrades on Chopin like every model we have, its micro equivalent is plausibly ~80–84 — i.e. around cyolo_sb_a, not clear of it.

3.3 The benchmark cannot support small claims [V]

Piece-level cluster bootstrap: **design effect ≈ 180×**. Effective N is **25 pieces, not 4415 onsets**. MDE ≈ 10 points; 99 pieces would give 5, 619 would give 2.

[V] Corroborated by a retrain control: 3 of 6 identical-recipe retrains significantly beat *themselves*. B4 scored 22.7 in one run and 16.4 in another, d = −6.29 — **larger than a typical published ablation effect**.

3.4 OMR on clean engraving is close to solved [V]

`OMR_FEASIBILITY.md` + job 452178. 20 MSMD pages, two render resolutions, scored against MUNG ground truth at 0.5-notehead-width tolerance:

variant	precision	recall	F1	median $\|\Delta x\|$	p90 $\|\Delta x\|$
native	0.9902	0.9807	**0.9854**	0.33 px (0.033 nhw)	0.79 px
hires	0.9970	0.9881	**0.9925**	0.31 px	0.78 px

Per-page F1 spread: min 0.949, median 0.985, max 0.996 — **no catastrophic page**. Spurious detections 0.30–0.98%. Cost ~5 min/page.

This substantially beats the [E] estimate (~97–99% detection, which held; but the coordinate precision — a third of a pixel — is far better than assumed). Failure taxonomy is orderly: 4-ledger-line notes miss at 6.67% (5.6× base rate), dense neighbourhoods and chord clusters next; horizontal position on the page is essentially neutral. The domain match is exact — **[V]** DeepScores was "generated by rendering existing MusicXML files with **Lilypond**", the same engraver MSMD uses, so detectors trained on it are in-domain here.

3.5 External real-audio benchmarks barely exist [V + A]

`EXTERNAL_BENCHMARKS.md`. This corrected two claims I had previously relayed:

- **[V] SMR's released audio is synthetic.** The public release ships only `data/midi/` and `data/pdf/`; `01_prepData.ipynb` cell 3 renders it with `fluidsynth -F p{i}.wav default.sf2 .../midi/p{i}.mid`. The real-audio version (200 YouTube recordings + manual line timestamps) is **unreleased**. Drop it.
- **[V] MAcc is not our metric.** `mus_align/eval.py` compares fractional **measure indices** with `error_boundary=0.5` on a 100 Hz time grid — **[A]** ≈ ±1.10 s at MeSA-13's 2.20 s median measure, ~2× looser than pct@0.5s, and time-weighted rather than onset-weighted.
- **[A] MeSA-13 is the only dataset pairing real scans + real audio + fine time alignment that exists** — and its 13 pieces power only a ~14-point claim, *worse* than our current 25. It is a validation set, not a benchmark. Its fully-automatic baseline is **0.72**, not the 0.82 headline (which is human-in-the-loop).
- **[A]** Shelving ASAP/Magaloff/Zeilinger was correct: Matchmaker benchmarks them on symbolic scores with **no sheet images**, so they cannot score a pixel-position model.
- **[A]** YTSV/U-MusT has **762 h of real solo-piano audio** — useless for evaluation (slide-level alignment) but the most relevant corpus found for real-audio *pretraining*.

3.6 Bugs found and fixed along the way [V]

bug	consequence if unfixed
`impulse_response.py` used `fftconvolve(mode='same')`	4.1–20.0 frame label desync on 50% of augmented samples
`_frame()` floored instead of ceiling	dropped up to hop−1 trailing samples per file
`eval_all_tiers.sh` used `dependency=afterok`	cancelled all 6 chained evals (training TIMEOUTs by design)
cyolo `world_size` AttributeError; fork-pool deadlock	training would not start (OpenMP is not fork-safe with `Pool(8)`)
onnxruntime ignores `OMP_NUM_THREADS`	oemer workers spawned 64 threads each, OOM-killed at rc=137
`shift_diag` fed negative onsets to mir_eval	an *optional diagnostic* killed the primary measurement

---

4. What this adds up to: two deliverables

4.1 Deliverable A — the evaluation paper (analysis complete)

This is the contribution that is **already earned**. Nothing in it depends on beating anyone.

Claims, all backed above: 1. The SOTA claim is measured under an **undisclosed oracle**: CODA consumes ground-truth system and bar boxes at train and test, which CYOLO must predict (§3.1).

2. It is also measured with a **different aggregator**, worth **+7.3 points** on our data — comparable to CODA's entire claimed margin (§3.2). 3. The benchmark's effective N is **25, not 4415**; MDE ≈ 10 points, and identical-recipe retrains differ by more than a published ablation (§3.3). 4. **Constructive half:** report micro *and* macro; report piece-level bootstrap CIs; report the acoustic tier grid (`synth` / `rp_synth` / `di-left` / `room`) rather than one number; and disclose which structures a model is *given*.

Risk to be honest about: this is a critique paper. It needs the constructive half (4) and ideally a model result to avoid reading as sour grapes.

4.2 Deliverable B — the model track

Target: **79.9**. Current: **45.6**. The §2 result says the audio side is healthy, so effort goes to the output parameterisation and the temporal decoder.

---

5. Priorities, in order

P1 — Output parameterisation (highest expected value)

Why first. §1.2 gives a 36-point same-lab swing from exactly this change (MM-Loc 58.5 vs CUNet 22.4). §2 independently says the failure is not the audio. Two unrelated lines of evidence converging on the output layer is the strongest signal in this document.

Do: replace the dense soft-Dice heatmap with a ranked/classification output — bucketed softmax over x position (MM-Loc style), or box + objectness (CYOLO style, = H2 in `HYBRID_MODELS.md`).

Prediction: room +10–20. **Falsifier:** if it moves room by <5 with synth unchanged, the output layer is not the bottleneck and P2 becomes primary.

P2 — MERT audio tower inside CYOLO's detector (H1)

The two largest effects we have measured live on opposite sides and have never been combined: MERT is worth +22 on room; the detection formulation degrades −9.4 synth→real where our dense heatmap degrades ∼−40.

Blocked on one thing. [V] `train_cyolo_repro_gpu.sh:98-106` passes `--train_sets --val_sets --config --augment --dump_root --log_root --tag --num_workers` and **no IR flag** — so our reproduction reproduces the published **no-IR** row. Henkel & Widmer (EUSIPCO 2021) report CYOLO **without** IR = 46.0 room and **with** IR = 71.2 (+25.2). We must confirm whether their `train.py` exposes an IR path and relaunch with it, or we are modifying the wrong base. **Zero API cost — pure cluster work.**

Prediction: room ≥ 60. **Falsifier:** if room lands near plain cyolo_sb with synth unchanged, MERT and the detector are redundant rather than complementary.

P3 — The online matcher (newly opened by §2.2)

Greedy loses the pointer and sits 8–13 s away from truth. A causal monotonic matcher with re-entry — this is what `M1.md` was designed for — is now a *measured* gap rather than a speculative one. Note this is the same failure mode as the repeat-ambiguity diagnostic.

P0 — IR augmentation, correctly, on the MERT base (top priority) do this first

(This section replaces an earlier draft that de-prioritised IR augmentation on the strength of §2. That reasoning was wrong and is corrected here.)

Why §2 does not argue against IR — it argues FOR it. "The room does not destroy the information" and "our model is not invariant to the room" are different statements. AMT proves the signal survives; our model still fails because it trained only on dry synthetic audio and its features shift out-of-distribution. That is exactly the failure augmentation fixes — and §2 is what guarantees there is recoverable signal to become invariant *to*. Had the room destroyed the information, no augmentation could help.

Three facts that make this the highest-value action in the document:

1. **[V]** We sit almost exactly on CYOLO's published **no-IR** row: ours 81.1 synth / **45.6** room / 56% retention; CYOLO-no-IR 80.4 / **46.0** / 57%.
2. **[A]** Henkel & Widmer (EUSIPCO 2021) report the IR delta as **46.0 → 71.2 = +25.2 points** on room.
3. **[V] IR augmentation has never been correctly tested on our models.** `extensions/augmentation/impulse_response.py` used `fftconvolve(mode='same')`, which advances audio by `(len(ir)-1)//2` samples — a **4.1–20.0 frame label desync on 50% of samples**. Fixed in `e8320ea` (**2026-08-06**). B6, our only IR run, trained **2026-07-05 – 07-08** — a month *before* the fix. B6 came last on room (15.6) and we concluded IR augmentation does not work for us. **We were reading a bug.**

[V] CYOLO gates IR separately from `--augment`: `train.py:241` defines `--ir_path` (default `None`) and `dataset.py:317-319` builds `ImpulseResponse` only when it is set. Our `train_cyolo_repro_gpu.sh` never passes it — so our reproduction has tempo and image-shift augmentation but **no IR**, which is why it lands on the no-IR row. (An in-script comment claiming `--augment` includes IR convolution is **wrong** and should be corrected.)

[V] Assets are ready: **270 impulse responses** in `/scratch/pmohseni/ir_bank/mit_ir_survey/Audio/`, plus an `openair/` set.

Two runs, both cluster-only, no API cost: - **P0a** — our MERT base retrained with the *fixed* IR augmentation. - **P0b** — `cyolo_sb` relaunched **with** `--ir_path`, giving us the correct base for P2 and a verified reproduction of the 71.2 row.

Prediction: P0a room **60–70**. **Falsifier:** if it lands under 55 with synth unchanged, the published IR delta does not transfer to a dense-heatmap model, and P1 becomes the whole plan.

---

6. Explicitly ruled out

direction	why
SMR as a benchmark	**[V]** its released audio is fluidsynth-synthetic
ASAP / Magaloff / Zeilinger / Batik / Vienna4x22	**[A]** symbolic scores, no sheet images — cannot score a pixel model
CollabScore	**[A]** contains no audio at all; it is an OMR ground-truth set
SMT / Zeus / homr for OMR	**[A]** better transcribers, but structurally discard pixel coordinates
Chasing CODA's 88.3 head-on	it is macro + oracle-boxed; the honest target is 79.9
More CB_TA-Ext variants	no non-detection model in the family has exceeded 58.5; the ceiling is structural

---

7. The decomposed pipeline: what it is and is not

A standing scope question, recorded so it does not drift again.

It is a diagnostic. Audio → AMT notes ⟷ score → OMR notes → position is **offline alignment**, a different task from online score following. §3.4 shows OMR is not the blocker (F1 0.985–0.993, sub-pixel Δx) and §2 shows AMT is not either — so the decomposition *would* work, and that is precisely what makes it useful as a measurement: **it proves the information is present and localises the failure to our architecture.**

It is not the paper. Pivoting to it would abandon online score following. The OMR and AMT results earn **one table row each** as diagnostics, and the effort goes back to §5.

---

8. Open questions

1. **[V, unresolved]** Does CYOLO's `train.py` expose an IR augmentation path? Gates P2. Cheap to answer.
2. **[E]** Magnitude of CODA's oracle-layout advantage. Unbounded from code — CODA cannot run without boxes and none of its five ablations removes them.
3. **[A]** Whether CYOLO's *published* table used `--only_onsets`. Our reproduction does and matches exactly, which is strong evidence it did.
4. **[A]** MeSA-13 redistribution terms — the repo has **no LICENSE file**, and audio provenance is mixed. Fine to evaluate and cite; not to redistribute.
5. Does the AMT result hold on a *human* performance? The MSMD-Rec recordings are Disklavier playback with no tempo deviation, which is an easier setting.

---

9. Immediate next actions

#	action	cost	blocks
1	**P0a** — retrain MERT base with the *fixed* IR augmentation (270 IRs ready)	cluster only	the whole model track
2	**P0b** — relaunch `cyolo_sb` with `--ir_path` (flag confirmed to exist)	cluster only	P2
3	**P1** — bucketed-softmax / objectness output head on the MERT base	cluster only	—
4	Fix the wrong `--augment`-includes-IR comment in `train_cyolo_repro_gpu.sh`	trivial	—
5	Draft the evaluation paper — §4.1 is complete and pushed	writing	—
6	Fold the OMR + AMT diagnostics in as one table row each	writing	—

No further research agents are queued. The remaining work is building and writing.

The arithmetic to 79.9

step	room	basis
today	45.6	measured
+ P0a (fixed IR augmentation)	60–70	**[A]** published delta +25.2, discounted for transfer
+ P1 (output reparameterisation)	+5–15	**[V]** MM-Loc vs CUNet = +36 same-lab, discounted hard
target	**79.9**	cyolo_sb, **[V]** self-reproduced

P0a and P1 are independent — one changes the input distribution, the other the output layer — so they should partially stack. They are not additive, since both attack the same synth→real degradation. **Reaching 79.9 needs both to land near the top of their ranges**, or P2 (MERT-in-CYOLO) on top. Honest read: 65–75 is likely, 80 is reachable but not assured.

---

10. Document map

file	holds
`BASELINE_CORRECTION.md`	the 17-point column error; corrected scoreboard
`AGGREGATION_FINDING.md`	micro vs macro, measured on our data
`CODA_COMPARABILITY.md`	full CODA audit with file:line evidence
`OMR_FEASIBILITY.md`	OMR survey + the measured oemer result
`EXTERNAL_BENCHMARKS.md`	MeSA-13 / SMR / everything ruled out
`HYBRID_MODELS.md`	H1–H4 designs (numbers superseded; mechanisms stand)
`M1.md`	monotonic alignment — now motivated by §2.2
`results/analysis/`	power, calibration, OMR scored JSON

Baseline correction (2026-08-04) — the target was wrong by 17 points

Verified directly, not taken from an agent's word.

What was wrong

Throughout the real-audio work the bar was quoted as `cyolo_sb = 63.0` and `cyolo_sb_a = 70.6` on MSMD-Rec `room`. **Those are the ≤0.1 s column of Frontiers Table 4, not ≤0.5 s** — which is the metric every one of our own numbers uses.

Proof, both independent:

1. `cyolo_score_following/eval.py:65` hardcodes `thresholds = [0.05, 0.1, 0.5, 1.0, 5.0]` — so column 3, not column 2, is 0.5 s.
2. Our own reproduction, `results/eval_cyolo_sota-66914671.log`, prints for cyolo_sb_a on room: `<=0.05: 68.2 / <=0.1: 70.6 / <=0.5: 86.5 / <=1.0: 89.1 / <=5.0: 98.1`, matching published `0.682/0.706/0.865/0.891/0.981` exactly. 70.6 is unambiguously the 0.1 s cell.

Corrected table (pct@0.5s, MSMD-Rec room)

model	room @0.5s	previously quoted
CYOLO	**71.2**	58.1 (was the 0.1s cell)
CYOLO-SB — the reproducible bar	**79.9**	63.0 (was the 0.1s cell)
CYOLO-SB+A (not reproducible, needs "+A" data)	86.5	70.6 (was the 0.1s cell)
CUNet — published, our own architecture family	**22.4**	never quoted
MM-Loc (Dorfer-style bucketed softmax)	58.5	never quoted
our R3 (MERT + pitch-aux + belief filter)	**45.6**	—

The gap to beat is therefore **34 points (45.6 → 79.9)**, not 17.

Two things this reframes

We are not failing — we are extending a weak baseline. Published CUNet, the exact dense-heatmap + Dice family we build on, scores **22.4** on room. R3 is 45.6. We have already roughly **doubled** the published version of our own architecture. That was never visible while the comparison was against the wrong column.

CYOLO's "4.3-point synth→real drop" is only the non-reproducible +A model. Reproducibly, at 0.5 s: CYOLO 88.5→71.2 = −17.3; CYOLO-SB 89.3→79.9 = −9.4. Ours is ~−40. Still a large gap, but not the 45-vs-4.3 chasm that framed the earlier track design.

The strongest lead this surfaced

A **within-paper, same-lab, same-data** controlled comparison in Frontiers Table 4: **MM-Loc = 58.5** on room vs **CUNet = 22.4**, with essentially the same audio tower. A 36-point swing attributable to the OUTPUT PARAMETERISATION (bucketed softmax classification over position vs dense soft-Dice heatmap). This is much stronger evidence than anything in HYBRID_MODELS.md for attacking the loss/output formulation first. It is corroborated by the OOD-segmentation literature (Galdran et al.) finding soft-Dice wins in-distribution and loses out-of-distribution.

Bug found in passing

`extensions/augmentation/impulse_response.py:85` uses `fftconvolve(waveform, ir, mode='same')`, which shifts the signal by ~half the IR length **relative to its onset labels**. B6's catastrophic 15.6 on room is therefore not evidence that IR augmentation fails — the experiment is invalid.

Confined to that one file: `scripts/precompute_mert_augmented.py` (the R2 bank) uses `mode='full'` then truncates, which is correct. R2's 6615-piece degraded bank is unaffected.

Claim that did NOT survive checking

An agent reported that all CB_TA-Ext models were trained without tempo augmentation, citing `my-model/cpjku_adapter/train_official.py:305` (`'tempo_factors': [1000]`). That is an older, separate code path. Our runs go through `third_party/cpjku_unet/audio_conditioned_unet/train_model.py` with `--config configs/msmd_aug.yaml`, which sets `tempo_factors: [500, 750, 950, 1000, 1050, 1250, 1500]`. Tempo augmentation is ON for the CB_TA-Ext line.

The SOTA claim compares macro-aggregated to micro-aggregated numbers

Verified independently on our own data, 2026-08-06.

The two aggregators

	code	what it computes
CPJKU / Henkel & Widmer / us	`eval_model.py:102` `np.sum(onset_diffs <= th) / total_onsets`	**MICRO** — every onset weighted equally
CODA (ISMIR 2026)	`scripts/evaluate_batch.py:183` `float(np.mean(ratios))`	**MACRO** — every *piece* weighted equally

What that is worth on this benchmark

Recomputed from our R3 room run's own per-page output (`results/eval_any-111639.log`, 25 pages, 4415 onsets):

aggregation	R3 room pct@0.5s
MICRO (pooled onsets) — what we and every baseline report	**45.4** *(log: 45.6; reconstruction sound)*
MACRO over 25 pages	46.8
MACRO over 16 pieces — CODA's aggregator	**52.7**

The aggregation choice alone is worth +7.3 points, on an unchanged model.

Why it is so large

One recording dominates, and it is the one every model fails:

- Chopin Nocturne Op.9: **1315 / 4415 onsets = 29.8% of the micro metric**
- but only **6/25 pages (24.0%)** macro-by-page, and **1/16 (6.2%)** macro-by-piece
- it scores **14.6** where the rest of the set scores **57.0**

So macro aggregation quietly down-weights the hardest 30% of the benchmark by ~5x.

Why this matters for the published claim

CODA reports **88.3** on MSMD-Rec real audio and compares it against CYOLO 71.2, cyolo_sb 79.9 and cyolo_sb_a 86.5 — numbers it **copied from Henkel & Widmer without re-running** (its own §4 says so). Those copied numbers are micro.

If CODA's 88.3 is macro and it degrades on the Chopin Nocturne the way every model in our table does, its micro equivalent is plausibly **~80-84** — i.e. around cyolo_sb_a, not 1.8 points clear of it.

The aggregation difference (+7.3 measured here) is comparable to CODA's entire claimed margin over cyolo_sb (+8.4) and larger than its margin over cyolo_sb_a (+1.8).

What is verified vs inferred

VERIFIED (high confidence): - Both aggregator implementations, read from source. - The +7.3 gap, recomputed from our own per-page log. - CODA's repo contains **no real-audio evaluation code at all** — a case-insensitive grep for `real_perf` / `msmd_rec` / `msmd_rp` / `setting ii` across every `.py`/`.yaml`/`.sh` returns nothing, and `run_full_pipeline.sh` hard-asserts exactly 94/66/28 synthetic pieces. The row that made it SOTA is the one row the release cannot reproduce. - No pretrained weights: empty `git tag`, empty `git lfs ls-files`, GitHub releases API `[]`, HF models API for the author `[]`. - "16-piece subset" and "25 pages" ARE the same set: collapsing `_page_N` on our 25 gives exactly the 16 names in `room_split.yaml`, page counts sum to 25, and total duration of the 16 room wavs is 0.3115 h = Henkel's published "0.31 h". **So 88.3 is on our exact audio.**

INFERRED (not proven): - That CODA's *Setting II* number specifically is macro. `evaluate_batch.py` is the only aggregator in the repo and it is macro — but since no Setting II code was released, this cannot be confirmed from the artifact.

Consequences

1. **Our own standings table is internally consistent** — all micro — so R3 45.6 vs cyolo_sb 79.9 remains apples-to-apples. The gap is real.
2. **Report both aggregations from now on.** R3 is 45.6 micro / 52.7 macro-16.
3. **This is the evaluation paper's opening result**, and it is far stronger than the one we had (cyolo_sb vs cyolo_sb_a being inside noise). It is also not an attack on one group: three consecutive papers on this benchmark report bare point estimates with no CIs, no multi-seed runs, and now a silent aggregator change.

Is CODA's SOTA claim comparable to CYOLO's? — a code-level audit

CODA (arXiv:2607.21899, ISMIR 2026, github.com/ValleyC/CODA @ `dba9829`, MIT) reports .743 vs CYOLO-SB's .630 at ≤0.10 s on real audio, and .893 vs .885 at ≤0.50 s on synthetic. This file records what the released code says about whether those two columns measure the same thing.

Audited against a clone at `/scratch/pmohseni/coda_audit/CODA`. Every claim below carries a `file:line` that was read directly. Claims marked **[inferred]** were not verifiable from code.

Headline: CODA is given the page layout; CYOLO has to find it

`coda/models/coda_model.py:314-317` — the boxes are **required positional arguments** of `forward()`, not an optional evaluation convenience:

```python
def forward(self, score, perf, system_boxes, bar_boxes, bars_per_system,
            gt_system_idx=None, gt_bar_in_sys=None,
            prev_gt_system_idx=None, prev_gt_bar_page_idx=None,
            tempo_aug=False, p_pred=0.0):
```

Chain of custody on the **training** path — this is the part that was not previously checked, and it is the part that matters:

step	location
read from MSMD npz	`coda/utils/data_utils.py:32` — `systems`, `bars`
into piece metadata	`coda/utils/data_utils.py:239-243`
into the sample	`coda/dataset.py:703-705`
collated	`coda/dataset.py:622-624`
into the train forward	**`scripts/train.py:173-177`**
consumed as ROIs	`coda_model.py:402-407` (system), `:482-489` (bar)

Boxes are never perturbed: `coda/dataset.py:711-728` applies only the same x/y roll already applied to the score image, so they stay pixel-registered.

The loss confirms there is nothing to learn about layout — `coda/utils/loss.py` `selection_loss` has exactly three terms: system CE over the **given** candidates, bar CE over the **given** candidates, and note MSE in bar-local coordinates. No box regression, no objectness, no IoU. The module says so itself (`coda_model.py:10`):

> No anchors, no NMS, no objectness – pure classification over known candidates.

and so does `configs/coda.yaml:2`:

> System -> Bar -> Note cascade via classification over known MSMD annotations.

No detection path exists in the release at all. `coda/models/backbone.py:60-62` explicitly skips any layer named `Detect` when building from YAML, and `configs/coda.yaml` contains none to skip. The inherited YOLO builder `coda/models/builder.py:29-33` (with its `no = na * 5  # bbox + objectness`) is **dead code** — `backbone.py:43` uses its own `_build_layers` instead. The `predict_sb=True` targets built at `dataset.py:576-578` never reach a loss; `data.targets` survives only so `selection_getitem` can recover the augmentation shift (`dataset.py:716`) and for video overlay.

All three consumers of boxes are oracle: training (`train.py:175-177`), streaming validation (coda/utils/streaming_eval.py:130-137), and reported evaluation (scripts/evaluate.py:514-516, 522-525).

So CODA's task is "pick 1 of N given regions." CYOLO's is "produce the region." The reported system accuracy .991 vs .963 and bar .975 vs .890 are measured across that asymmetry. This is not a tuning-level defect — the two systems solve different problems. The paper does not state it in §4.1.2 or §4.1.3; it is visible only in the code and in the config comment.

[inferred] The *magnitude* of the advantage cannot be bounded from code. CODA cannot be run without layout boxes, and none of Table 3's five ablations removes them.

Aggregation: CODA reports macro, CYOLO reports micro

CODA, per piece — scripts/evaluate.py:719:

```
onset_ratios[t] = sum(1 for e in onset_errors if e <= t) / len(onset_errors)
```

CODA, across pieces — scripts/evaluate_batch.py:179-183, unweighted mean of the 94 per-piece ratios, and the Table 1 LaTeX row consumes exactly these (evaluate_batch.py:357):

```
    for t in ONSET_THRESHOLDS:
        key = f'onset_ratio_{t:.2f}s'
        ratios = [m[key] for m in all_metrics if m.get(key) is not None]
        if ratios:
            summary[f'mean_{key}'] = float(np.mean(ratios))
```

CYOLO — eval.py:76-83, pool first, then divide:

```
    frame_diffs = np.concatenate([piece_stats for piece_stats in stats['piece_stats'].values()])
    total_frames = len(frame_diffs)
    ...
        ratio = np.sum(frame_diffs <= th) / total_frames
```

The per-piece print at eval.py:59-73 is display-only and never averaged.

The two LaTeX rows are therefore produced by **different estimators**. Our own measurement of one checkpoint across both — 45.4 micro vs 46.8 macro on 25 pieces (AGGREGATION_FINDING.md) — puts the pure estimator effect near +1.4, which does not explain CODA's 7.7-point Table 1 gap but is uncorrected and directionally favorable.

It matters most in Setting II, which is macro over just 16 pieces of very unequal length against a micro baseline. That is exactly the regime where our own macro-16 vs micro spread reached **+7.3** points. Treat `.743 vs .630` as the least trustworthy number in the paper.

Internal inconsistency worth noting: CODA computes both macro and micro for the jump table (evaluate_batch.py:195, 198, 214, 218) and Table 2 uses **micro** (evaluate_batch.py:307-310), while Table 1 uses macro. Two tables in one paper, two estimators, neither disclosed — §4.1.2 says only "the cumulative ratio of onsets tracked," which reads as micro.

Onset protocol: clean. Settled by our own reproduction.

Both sides use `[0.05, 0.10, 0.50, 1.00, 5.00]` **seconds** (evaluate.py:691, eval.py:57).

There was an open question about the denominator: CYOLO's --only_onsets (eval.py:18) defaults to False, and its output is labelled "Average frame tracking ratios" — so if the published table were produced by the default invocation, it would be all-frame while CODA's is onset-only.

Resolved. Our reproduction passes `--only_onsets` (`eval_cyolo_sota_cpu.sh:43,49`) and yields `cyolo_sb` = **63.0 @0.1 s / 79.9 @0.5 s**, matching CODA's copied baseline row `.630 / .799` exactly. The published CYOLO numbers are onset-only. The denominators agree and this is **not** a defect.

Useful side effect: it means our 79.9 bar is self-reproduced, not trusted.

Reproducibility of the release

- **No pretrained weights.** `README.md:231-233`: *"Model weights are not included in this source repository."* No `.pt`/`.pth`/`.ckpt`, no Git LFS. Even synthetic Table 1 needs a 50-epoch retrain to re-derive.
- **No real-audio path.** Setting II pairs synthetic images with real piano recordings on a 16-piece subset, but the release ships no such data, no split file, and no code path. `scripts/finalize_run.py:12-14` hard-codes `{"standard": 94, "repeat": 66, "random": 28}` and refuses to emit `status: complete` otherwise. The 16 pieces are never identified. **Setting II is not reproducible from this release.**
- **Baselines copied, not re-run.** §4.1.3: *"All baseline numbers are taken directly from [10] under the same evaluation protocol."* Table 1 caption: *"† Proprietary training data; baseline numbers from [10]."* [10] = Henkel & Widmer. No CYOLO submodule, checkpoint, or invocation in the repo; the only CYOLO figure in code is a hard-coded latency comment (`evaluate_batch.py:464`).

Two smaller one-sided effects

- **Truncated-tail onsets dropped from the denominator** (favorable, likely <1 point). `evaluate.py:421-423` breaks out of the frame loop when `to_ > signal_np.shape[-1]`; `evaluate.py:699-700` then `continue`s past those onsets rather than counting them as errors.
- **Time-conversion mechanics differ** (mixed sign, not quantifiable). CODA builds an x→time map **per system** and indexes it by the *predicted* system (`evaluate.py:342-350, 707-710`), assigning `time_err = 100.0` on a wrong-system prediction (`evaluate.py:713`) — a hard miss at every threshold, **harsher** than CYOLO's single unrolled page map (`dataset.py:194`), which can still land inside 5 s after a system error. Conversely `np.interp` clamps to the system's x-range, bounding within-system error by that system's duration rather than the page's — mildly **lenient**.

Ranked summary

#	issue	direction	size
1	Oracle system/bar boxes at **train and test**	favors CODA	large, **[inferred]** — unbounded from code
2	Macro (CODA) vs micro (CYOLO) aggregation	favors CODA	~1.4 pts on 94 pieces; **up to ~7 on Setting II's 16**
3	Setting II unreproducible; no weights released	unverifiable	n/a
4	Truncated-tail onsets dropped	favors CODA	<1 pt

#	issue	direction	size
5	Per-system interpolation + 100 s wrong-system penalty	mixed	small
—	Thresholds, units, onset-only denominator	**no defect**	—

What this means for us

The bar we must clear is **our own reproduced `cyolo_sb` = 79.9 @0.5 s**, not CODA's 88.3. CODA's margin over CYOLO is measured under an oracle-layout asymmetry that the paper does not disclose and a macro/micro estimator swap that it also does not disclose — and its real-audio claim, the one closest to our objective, sits in the regime where the estimator swap is worth the most.

Nothing here says CODA is a weak system. It says the published margin is not a like-for-like measurement, which is a result for the evaluation paper rather than an excuse for our numbers.

External real-audio benchmarks: what actually exists

Motivation. Our real-audio result rests on **25 pages**, and the piece-level cluster bootstrap gives a design effect of ~180x — effective N is 25, not 4415 onsets. That powers roughly a **10-point** claim (99 pieces → 5 points, 619 → 2). Our set also pairs *synthetic score images* with recordings of MIDI playback, so it is partly oracle-flavoured. An external benchmark with real scans, real audio and human annotations would fix both.

Claims are **[verified]** where I read the code/data myself, **[agent]** where they come from the survey and I did not re-check.

Correction to an earlier claim in `OMR_FEASIBILITY.md`

That file listed **SMR v1.0** as "200 solo piano IMSLP scans, one **real YouTube recording** each." **That is wrong. SMR's released audio is synthetic.**

[verified] The public release `HMC-MIR/YoutubeScoreFollowing` contains only `data/midi/` and `data/pdf/` (GitHub contents API) — no audio, no `timeAnnot/`, no `lineAnnot/`. Its own prep notebook renders the audio, `01_prepData.ipynb` cell 3:

```
audio_cmd = "fluidsynth -F p" + str(i) + ".wav default.sf2 " \
            "/home/mshan/ttemp/spring2020/data-v2.1/midi/p" + str(i) + ".mid -R=1"
```

[agent] The real-audio asset — 200 YouTube recordings with manual per-line timestamps — is a separate Shan & Tsai TISMIR 2021 addition and **is not published**; the notebooks reference `/home/mshan/ttemp/data/timeAnnot/` and `lineAnnot/`, which exist nowhere public.

So SMR is a *synthetic* benchmark, strictly worse for our purposes than the synthetic set we already own and understand. **Drop it.**

Correction: MAcc is not comparable to pct@0.5s

`MAcc 0.82` was quoted as though it sat on our axis. It does not.

[verified] `mus_align/eval.py`:

```
acc = np.sum(np.abs(ref_eval_measures - pred_eval_measures) <= error_boundary) / len(eval_times)
```

`error_boundary` defaults to **0.5**, and both sides are **fractional measure indices**, not seconds. The grid is uniform in *time*:

```
eval_times = np.linspace(0, pred.alignment.times[-1], int(pred.alignment.times[-1]/frame_rate))
```

at `frame_rate = 0.01` (100 Hz), and both interpolators use `fill_value='extrapolate'`.

Two consequences: - **[agent, measured from the 13 annotation files]** median measure = 2.20 s, so MAcc ≈ a **±1.10 s** criterion (per-piece 0.63–1.77 s) — roughly **2× looser** than our pct@0.5s. - MAcc is **time-weighted**; our pct@0.5s is **onset-weighted**. Silences, held notes and slow passages get duration weight instead of note count.

[agent] MAcc is also **macro** (unweighted mean over 13 pieces), whereas Shan & Tsai's line accuracy is **micro** (`total_acc / total_times`). Do not mix them — the same distinction was worth up to 7 points in `AGGREGATION_FINDING.md`.

MeSA-13 is the only real candidate that exists

[agent] 13 sheet-music scans + real performance audio + expert measure annotations. Public repo `https://github.com/mfeffer/mesa-13` (124 MB incl. 80 MB of demo videos; the 13 piece folders are ~62 MB) — easier than the CMU Box mirror. Three files per piece: score PDF, MP3/OGG of a real performance, and `alignment.json`:

```
{"audio_score_alignment": [
  {"audio_start": 9.43, "audio_end": 9.9, "bbox_number": 1,
   "measure_bbox": [285.08, 355.74, 600.36, 730.80], "page_number": 2}, ...]}
```

One entry per measure **in logical (performed) order** — repeated measures appear twice with the same bbox. Bboxes are absolute pixels at DPI 200.

pieces / annotated pages	**13 / 40**
measure instances / unique boxes	957 / 829
total real audio	**35.9 min**
median measure duration	2.20 s
systems (staff lines)	209
pieces with repeats / non-piano	2 / 2

License is the wart. No LICENSE file on the repo (GitHub API `license: null`). Code is MIT, papers are CC BY 4.0, the **data carries no stated terms**, and audio provenance is mixed (at least one IMSLP-hosted recording with its own per-file licence). Fine to download, evaluate on, and cite; **not** clean to redistribute a derived dataset.

"No oracle anywhere" needs qualifying. The README says annotations were "generated in a **semi-automated** fashion": madmom beat tracking + Waloschek measure detection produce heuristic boxes and timestamps, which musicians then audit and correct — 20 annotator-hours total. Good enough at 0.5–1 s tolerances, but the timestamps are corrected madmom output, not hand-placed.

Can our metric survive? No, not defensibly

The GT is a set of measure boundary times; JLTR builds the continuous map by **linear interpolation between them**. Within-measure position is *assumed*, not annotated. At a 2.20 s median measure, our 0.5 s tolerance is **23% of a measure** — well inside the rubato that linear interpolation cannot represent. Their ±0.5-measure choice is about the tightest defensible threshold given measure-level annotation.

Options, most to least defensible: 1. **Adopt MAcc as-is.** Directly comparable to published numbers; `evaluate()` is ~40 lines to reimplement. 2. **Time-domain pct@X s** with **X ≥ ~1.0 s**, labelled "% of *time*", not "% of onsets." 3. **Onset-weighted variant** using our AMT bridge's detected onsets as sampling points only (position GT still human). Non-standard; needs justification.

What is lost, plainly: note → measure is a ~2× loosening of tolerance plus a change of denominator from onsets to time. A model that wins on MAcc has *not* been shown to win at note-level localisation. The MeSA-13 authors say as much: "note-level alignment would be the most useful, but we posit that producing such alignments would be expensive."

Model-side gap is small: an adapter, not an architecture change

[agent] We need neither OMR nor page-layout handling — MeSA-13's ground truth *is* measure boxes with page numbers. Clustering them into systems is the same top-coordinate rule JLTR already uses, and it ran on the real annotations cleanly: **209 systems across 40 pages, no failures.**

	MeSA-13	our MSMD strips
system height (DPI 200)	263–497 px, median 365	120 px (fixed)
strip aspect ratio W/H	25–123, median 70	~153
measures per system	median 4 (range 1–8)	—

At our 120 px strip height MeSA-13 strips are ~8,300 px wide — **shorter** than our ~18,400 px MSMD strips. The architecture is not stressed.

Genuinely new: (a) input is a ~3× downscaled **scan**, not a LilyPond engraving — expect a real domain hit; (b) 2 pieces have repeats, so logical ≠ graphical order and any monotone left-to-right assumption fails them outright (this is exactly the M1 repeat-ambiguity problem); (c) 2 pieces are non-piano.

Baselines — and the one to actually quote

[agent] MeSA-13, macro MAcc over 13 pieces:

system	setting	MAcc	MErr (measures)
Shan & Tsai hierarchical DTW	automatic	0.33	10.9
JLTR (Bukey et al.)	**automatic**	**0.72**	1.9
JLTR + human repeat labels	R	0.82	0.4
JLTR + repeats + **GT measure boxes**	R,M	0.86	0.4
JLTR + repeats + boxes + clef/key	R,M,S	0.88	0.3

The fully automatic number is 0.72, not 0.82. The abstract's "33% → 82%" is the human-in-the-loop setting. And since we would consume the GT measure boxes to build the strip, the honest comparison row is **R,M = 0.86** — or report against 0.72 and disclose the boxes.

No CYOLO-family or CODA numbers exist on MeSA-13. JLTR positions Dorfer/Henkel (our lineage) as a *different task* — real-time, digital score. Everything reported there is offline DTW alignment. That is an opportunity (first score-following numbers on a real-scan benchmark) and a hazard (reviewers will ask why an online tracker is compared to offline DTW).

Power: MeSA-13 is a validation set, not a benchmark

Using our own calibration (25 clusters → 10 points, MDE ≈ 10·√(25/N)):

set	clusters	MDE (pts)
MeSA-13	**13**	**≈13.9**
our current real-audio set	25	10.0
ours + MeSA-13 pooled	38	8.1
SMR (*synthetic*)	100	5.0
Shan & Tsai real-audio set (**unreleased**)	200	3.5

13 pieces powers ~14 points — **worse than our current 25**. And that is optimistic: MeSA-13 is deliberately heterogeneous, and JLTR's own dispersion shows it (automatic MErr 1.9 measures, std **3.7**, i.e. std ≈ 2× mean).

Do not count the 40 pages instead (which would give 7.9). Pages within a piece share performer, recording, room and scan — piece-level clustering is right, for the same reason we argued it for our own set.

Everything else, ruled out

- **CollabScore — no audio at all.** 26 Saint-Saëns scores, Gallica images + MEI/MusicXML, CC BY-NC-SA. An OMR ground-truth set. The "links to audio fragments" description does not match what is released.
- **YTSV / U-MusT (arXiv:2505.12863)** — 12,317 videos, ~1,463 h real YouTube audio (**762 h piano solo**), score images from video slides, MIT code. But alignment is **slide/page-turn granularity, automatically derived** — far too coarse for our metric. **Interesting as a real-audio *pretraining* corpus for the domain gap, not as an evaluation set.** Did not appear in the earlier survey.
- **MUSCAT (ACM MM 2024)** — 80 h real audio, 1,251 scanned sheets, but annotations are score-level symbolic (for transcription), not time→pixel. Access by request.
- **ASAP / Magaloff / Zeilinger / Batik / Vienna4x22 / nASAP / PERiScoPe** — shelving these was correct. Matchmaker (arXiv:2510.10087) benchmarks on (n)ASAP, Batik, Vienna4x22 — all **symbolic/MIDI scores, no sheet images**. They cannot score a pixel-position model.
- **Sheet Music Benchmark (ISMIR 2025)** — OMR only, no audio.

MeSA-13 appears to be the only dataset pairing real scans + real audio + fine time alignment that exists. That is itself worth stating in the paper, and it is why it is small.

Recommendation

Pursue MeSA-13, but change what we want from it. It cannot power an improvement claim. What it *can* do, which nothing we own can, is support an **oracle-free existence claim**: "given only a real scan and a real recording, our score follower reaches MAcc X on the same 13 pieces where published automatic alignment reaches 0.33 and 0.72." One honestly-caveated table row that closes the "your real-audio result is partly oracle-flavoured" objection without claiming power it does not have. Report MAcc, macro, their exact `evaluate()`. **Do not report pct@0.5s on it.**

First concrete step (~1–2 days, 62 MB, no new dependencies, no GPU): 1. `git clone https://github.com/mfeffer/mesa-13` into `/scratch/pmohseni/` (skip `videos/`). 2. Write a MeSA-13 → strip adapter: render each PDF at DPI 200, read `alignment.json`, cluster GT boxes into systems by top coordinate, crop and concatenate into a 120 px strip, emit `strip_to_page_mapping` in our existing schema plus a per-measure `(measure_idx, strip_x_start, strip_x_end, audio_start, audio_end)` table. **Traverse in logical order** so the 2 repeat pieces unroll correctly — the only nontrivial part. 3. Reimplement `evaluate()` (~40 lines, no TF, no madmom); sanity-check by scoring the ground truth against itself — must give exactly 1.0. 4. Run our best real-audio checkpoint zero-shot. Expect a large drop; scans at 3× downscale are out of domain. **Measuring that drop is the informative outcome** — it tells us whether the 25-page result generalises at all.

Worth doing in parallel: the only properly-powered external real-audio set (200 clusters, 3.5-point MDE) is Shan & Tsai's YouTube annotations, which are unreleased. Email TJ Tsai (HMC-MIR) and ask for `timeAnnot/` and `lineAnnot/`. Short email, large payoff, and their code is MIT so they are release-friendly — the annotations look simply to have been left on a lab machine.

Can the decomposed pipeline stop being an oracle? — OMR feasibility

The decomposed pipeline (audio → AMT notes ⟷ score notes → pixel x) currently uses MSMD's LilyPond provenance to get notehead pixel coordinates for free. That makes it a **diagnostic**, not a method. This file records whether a released OMR system can recover the same thing from the page image alone.

Verdict: YES, viable now — conditional on choosing the weaker of two formulations of "note event" (see *The one real caveat*).

Claims below are marked **[verified]** where I read the source or the paper myself, **[agent]** where they come from the survey and I did not re-check.

The blocker was requirement 2, and it is satisfied

The pipeline needs, per notehead: (1) the note event, (2) **its pixel coordinate on the page**, (3) simultaneity / reading order across the grand staff.

Requirement 2 is what most recent OMR drops. The accuracy headlines all come from the end-to-end seq2seq family — SMT/SMT++, Zeus/olimpic, Polyphonic-TrOMR, and homr's second stage — which map image → token sequence and discard spatial grounding entirely. Reading the literature top-down leads to the wrong conclusion, because the systems that keep coordinates are the less fashionable pipeline/detection ones.

[verified] `oemer` (MIT, weights auto-download) keeps them. `oemer/notehead_extraction.py` defines on `NoteHead`:

attribute	meaning
`bbox: BBox`	**original-image pixel coordinates** — confirmed by `region = symbols[bbox[1]:bbox[3], bbox[0]:bbox[2]]`
`staff_line_pos: int`	vertical staff position; zero index at D4, may be negative
`track`, `group`	which staff / which system, assigned from the parent staff

Exposed as `oemer.layers.get_layer('notes')` / `'note_groups'`.

[agent] Also coordinate-preserving: **Audiveris** (AGPL-3.0 — flag the copyleft) via `Inter` bounds in the `.omr` XML; and the **Tsai / HMC-MIR bootleg-score** line (MIT), which is classical OpenCV with *no learned weights at all*, milliseconds per page.

Domain match is exact, not transfer

[verified] DeepScoresV2, ICPR 2020, §II:

> "DeepScores [9] is a huge synthesized dataset of typeset music… **The dataset was generated by rendering existing MusicXML files with Lilypond into annotated SVG images.**"

MSMD is LilyPond-engraved from Mutopia sources. **Same engraver.** So a DeepScores-trained notehead detector run on MSMD is in-domain, and the failure modes that dominate every reported

OMR degradation — scanner noise, historical typefaces, skew, bleed-through — are all absent from our case.

That degradation is large and worth stating: SMT++ Table 6 reports oemer at SER 90.3 and Audiveris at SER 94.6 on *scanned historical* piano. Both collapse off-domain. We never leave the domain they were built for.

[verified] DeepScoresV2's annotations also already carry, per the abstract, "higher-level rhythm and pitch information (**onset beat for all symbols and line position for noteheads**)" — i.e. requirements 1 and 3 natively, not just boxes. Noteheads are even split into `-InSpace` / `-OnLine` classes to make staff position robust.

[verified] Incidentally, that same paper lists MSMD as a peer dataset: "a medium, synthetic dataset of nearly 500 pieces… with aligned note-head annotations between the score image and the corresponding MIDI file."

Precedent: this exact pipeline already works on *harder* input

[agent] Tanprasert et al. (ISMIR 2019) and the follow-on Tsai-group work run image → notehead coordinates → symbolic alignment against MIDI, reporting **97.3% @1 s on 68 real IMSLP piano scans**, and **79.0%** line-level against real YouTube audio (MAcc 0.82 on MeSA-13).

They achieve that from a representation that throws away accidentals, key signatures, durations, and octave-exact pitch — quantizing x-position within each measure into 48 bins and building a binary staff-position × x matrix, so simultaneity becomes "same column." Deliberately weak, and robust *because* it doesn't try to be correct about voicing.

We would be feeding a cleaner detector on cleaner input.

The one real caveat

"Notehead + x-coordinate + staff-line position" is much easier than "note event with exact MIDI pitch + voice + simultaneity."

- **Staff-line position** is geometric — recoverable from bbox-y plus detected staff lines, about as accurate as the detection itself.
- **True MIDI pitch** needs clef + key signature + accidental scoping. This is where oemer's reported 91% → 77% degradation with score complexity comes from; ledger-line notes are a documented weakness.

The published successful systems chose the weaker formulation *and degraded the MIDI side symmetrically to match*. That is the design to copy, and it is what makes this YES rather than PARTIAL.

[agent, estimate not measurement] On clean LilyPond piano pages: ~97–99% of noteheads detected with correct pixel coordinates; ~90–95% correct staff-line position; ~80–90% correct true MIDI pitch; simultaneity/voice weakest at ~70–85%. Since DTW / monotonic alignment is a global-consistency method that tolerates 20–30% local error, every one of those clears the bar.

Recommended build

1. **Cheapest defensible non-oracle result:** port the bootleg-score extraction from `HMC-MIR/YoutubeScoreFollowing` (MIT) — OpenCV blob detection for filled noteheads, Hough/projection for staff lines and barlines, keeping pixel x. No weights, no inode cost,

milliseconds per page. Feed our AMT output where they feed MIDI. This yields the ablation that matters: **oracle coordinates vs OMR-recovered coordinates, same aligner.**

2. **If we need real pitch** (e.g. to use AMT pitch confidences): oemer, dumping `get_layer('notes')` alongside the MusicXML. Budget 3–5 min/page → a multi-GPU-day one-off over MSMD's ~500 pieces; cache it.
3. **Do not** reach for SMT / Zeus / homr. Better transcribers, structurally incapable of requirement 2.
4. Audiveris is the fallback, but AGPL-3.0 matters if anything ships, and it needs an `.omr` XML parser.

The measurement that should happen first

Nobody has published oemer or Audiveris accuracy on **clean engraved piano grand staff**. Every number available is either printed monophonic/violin or scanned historical piano. The 80–90% pitch figure above is an interpolation.

Run oemer on ~20 MSMD pages and score it against the LilyPond ground truth we already have. One afternoon, settles the estimate exactly, and the comparison is itself a publishable table — it would be the first measurement of OMR accuracy on this domain. `/scratch` has ~467k inodes free, enough for the venv.

External real-audio validation sets (separate, possibly bigger)

These do not have OMR-derived coordinates — they have *human* ones, which is strictly better ground truth — but they are real audio on real scans, which is exactly the regime where we lose ~34 points.

dataset	contents	granularity	license/access
MeSA-13 (ISMIR 2024)	13 scans + **real performance audio**, semi-automated measure bboxes + per-measure audio timestamps	measure	code MIT (`irmakbky/jltr-alignment`); data via `mfeffer/mesa-13`, **no LICENSE file**
~~SMR v1.0~~	~~real YouTube recordings~~ — **WRONG, see below**	—	—
CollabScore	~~links elements to audio fragments~~ — **contains no audio at all**; it is an OMR ground-truth set	—	`collabscore/dataset`

> **Corrections (verified 2026-08-07) — see `EXTERNAL_BENCHMARKS.md`. SMR's released audio is synthetic**, not real: the public release ships only `data/midi/` and `data/pdf/`, and `01_prepData.ipynb` cell 3 renders it with `fluidsynth -F p{i}.wav default.sf2 .../midi/p{i}.mid`. The real-audio version (200 YouTube recordings + manual line timestamps) is unreleased. **Drop SMR. MAcc is not our metric**: `mus_align/eval.py` compares fractional **measure indices** with `error_boundary=0.5`, on a 100 Hz uniform *time* grid — ≈ ±1.10 s at MeSA-13's 2.20 s median measure, so ~2× looser than pct@0.5s, and time-weighted rather than onset-weighted. The fully **automatic** JLTR baseline is **0.72**, not 0.82 (0.82 is human-in-the-loop with repeat

labels).

These change our output granularity from note to measure, so this is not a free substitution. And MeSA-13's 13 pieces power only a ~14-point claim — **worse than our current 25**. It is a validation set, not a benchmark.

Open

- **[agent]** No downloadable DeepScoresV2 detector checkpoint found (training code exists at `tuggeluk/mmdetection@DSV2_Baseline_FasterRCNN`). MuSViT releases only the MAE encoder on HF, not the DSV2 detection head — so its 97.0 mAP50 is a paper claim we would have to reproduce by fine-tuning.
- **[agent]** MeSA-13 redistribution terms and whether score images are included (behind a Box link, not fetched).
- The agent installed nothing and ran no code, per the inode/login-node constraints.